

Perfect You already have the `public_key` object from your `PRIVATE_KEY`. In Rails (really in plain Ruby with OpenSSL), you can extract the **modulus (`n`)** and **exponent (`e`)** directly, then encode them for JWKS.

Here's the clean way:

Extracting `n` and `e` from RSA public key

```
require "base64"
require "openssl"

public_key = PRIVATE_KEY.public_key

# Helper for base64url without padding
def base64url_uint(num)
  bin = num.to_s(2) # convert OpenSSL::BN to binary string
  Base64.urlsafe_encode64(bin, padding: false)
end

n_value = base64url_uint(public_key.n)
e_value = base64url_uint(public_key.e)

puts "n: #{n_value}"
puts "e: #{e_value}"
```

Example output

For a typical RSA 2048-bit key:

```
{
  "n": "pXhFrb7jEOv15HcFOwVa3vJhlYyy4zWvT3OiTpnqJDxNxRhLcDLvmCsHSrgidImkc
  "e": "AQAB"
}
```

Why this works

- `public_key.n` → modulus(`OpenSSL::BN`)
- `public_key.e` → exponent(`OpenSSL::BN`)

- `.to_s(2)` → gives raw bytes (binary string)
 - `Base64.urlsafe_encode64(..., padding: false)` → encodes to the JWK-required base64url form
-

This is the **Rails-friendly way** to build your JWKS `n` and `e` values from your stored RSA key.

Do you want me to also show you how to assemble the **entire JWKS hash** (with `key`, `alg`, `use`, `kid`, `n`, `e`) directly from this?